

Tarea: Análisis de Sentimientos con Redes Neuronales Recurrentes

1. Introducción

Esta tarea tiene como objetivo principal guiar a los estudiantes en la construcción, entrenamiento y evaluación de un sistema de **Análisis de Sentimientos** utilizando **Redes Neuronales Recurrentes (RNNs)** en el marco de PyTorch. A través de este ejercicio práctico, se explorará cómo estas arquitecturas son capaces de procesar secuencias de texto para clasificar reseñas de películas como positivas o negativas. Se enfatizará la importancia de cada fase del desarrollo, desde la preparación de los datos hasta la interpretación de los resultados, fomentando la justificación de las decisiones de diseño y la reflexión crítica sobre el proceso.

Las RNNs apiladas, también conocidas como **RNNs profundas**, consisten en múltiples capas recurrentes dispuestas una encima de la otra. Cada capa superior toma la secuencia de estados ocultos de la capa inferior como su entrada, permitiendo que el modelo aprenda representaciones jerárquicas y más abstractas del lenguaje a medida que la información fluye a través de las capas. Esta capacidad de construir representaciones de características de alto nivel es crucial para tareas complejas de Procesamiento de Lenguaje Natural (PLN) como el análisis de sentimientos.

2. Objetivo del Laboratorio

Construir, entrenar y evaluar un sistema de Análisis de Sentimientos basado en Procesamiento de Lenguaje Natural (PLN) utilizando la biblioteca PyTorch. Los alumnos deberán implementar una arquitectura de Red Neuronal Recurrente (RNN) para clasificar reseñas de películas como positivas o negativas, justificando las decisiones de diseño en cada fase del desarrollo.

3. Instrucciones Detalladas

Paso 1: Adquisición y Limpieza de Datos

La calidad de los datos de entrada es fundamental para el rendimiento de cualquier modelo de aprendizaje automático. En esta fase, se preparará el texto crudo para su posterior procesamiento.

- Descarga del Dataset:** Adquieran el dataset IMDB. Se recomienda utilizar las herramientas de datasets de Hugging Face (`'datasets'` library`) o importar los archivos en formato tabular si ya se tienen localmente. Asegúrense de tener acceso a los conjuntos de entrenamiento, validación y prueba.
- Función de Limpieza de Texto:** Desarrollen una función robusta para preprocesar el texto crudo de las reseñas. Esta función debe incluir, como mínimo, las siguientes operaciones:
 - Eliminación de Etiquetas HTML:** Las reseñas web a menudo contienen `'tags'` HTML` (ej. `'
'``). Utilicen expresiones regulares para removerlos y evitar que sean interpretados como parte del texto.
 - Conversión a Minúsculas:** Normalicen todo el texto a minúsculas para tratar palabras como "Excelente" y "excelente" como el mismo token, reduciendo la dimensionalidad del vocabulario.
 - Eliminación de Puntuación y Caracteres No Alfanuméricos:** Remuevan signos de puntuación, números y cualquier carácter especial que no aporte significado semántico al análisis de sentimientos. Las expresiones regulares son herramientas poderosas para esta tarea.
 - Tokenización:** Dividan el texto limpio en unidades más pequeñas (tokens), que generalmente son palabras. Pueden optar por una tokenización simple basada en espacios o utilizar librerías más avanzadas como NLTK para una tokenización más sofisticada.

Paso 2: Procesamiento y Preparación de Secuencias

Una vez limpios, los tokens deben ser convertidos a un formato numérico que el modelo pueda entender y procesar.

- Construcción del Vocabulario:** Creen un diccionario de mapeo bidireccional (`'word2idx'` e 'idx2word'`). Es crucial limitar el tamaño máximo del vocabulario para gestionar la complejidad computacional y mitigar el ruido de palabras poco frecuentes. Filtren las palabras con muy baja frecuencia (ej. aquellas que aparecen menos de 'N'` veces). Reserven índices numéricos específicos para tokens especiales:
 - <PAD> (índice 0): Utilizado para rellenar secuencias y asegurar longitudes uniformes.
 - <UNK> (índice 1): Representa palabras "desconocidas" o "fuera del vocabulario" (Out-Of-Vocabulary, OOV) que no se incluyeron en el vocabulario. Es importante que el vocabulario se construya exclusivamente con el conjunto de entrenamiento para evitar la fuga de información (data leakage) del conjunto de prueba.`

1. **Padding y Truncating:** Transformen cada reseña (ahora una lista de tokens) a una secuencia de índices enteros utilizando el `'word2idx'` creado. Definan una `'longitud máxima'` para todas las secuencias. Las reseñas más largas que esta longitud máxima deberán ser **recortadas** (truncating), mientras que las más cortas deberán ser **rellenadas** (padding) con el token `'<PAD>'` hasta alcanzar la longitud uniforme. Esto es esencial para poder agrupar las secuencias en `'batches'` y procesarlas eficientemente por la RNN.
2. **Datasets y DataLoaders:** Implementen una clase personalizada que herede de `'torch.utils.data.Dataset'`. Esta clase encapsulará las secuencias preprocesadas y sus etiquetas correspondientes. Luego, configuren instancias de `'torch.utils.data.DataLoader'` para los subconjuntos de entrenamiento, validación y prueba. Los `'DataLoaders'` facilitan la iteración sobre los datos en `'batches'`, la mezcla (shuffling) y el paralelismo en la carga de datos.

Paso 3: Definición del Modelo de Deep Learning

En esta sección, se diseñará la arquitectura de la Red Neuronal Recurrente Apilada.

1. **Implementación del Modelo:** Creen una clase en PyTorch que herede de `'nn.Module'` para definir su modelo clasificador. La arquitectura debe incorporar los siguientes componentes:
 - **Capa `'nn.Embedding'`:** Esta capa aprenderá representaciones densas (embeddings) para cada token en el vocabulario. Transformará los índices enteros de los tokens en vectores de números reales de alta dimensionalidad, capturando relaciones semánticas entre palabras.
 - **Capa Recurrente Apilada:** Utilicen una capa recurrente como `'nn.LSTM'` o `'nn.GRU'`. Es fundamental que sea **apilada** con al menos **2 capas de profundidad** (`'num_layers >= 2'`). Definan adecuadamente las dimensiones del estado oculto (`'hidden_size'`) para cada capa. Las capas apiladas permiten que el modelo capture dependencias temporales más complejas y aprenda representaciones jerárquicas del texto.
 - **Capas de Regularización (`'nn.Dropout'`):** Incorporen capas `'nn.Dropout'` después de la capa de `'Embedding'` y entre las capas recurrentes (si aplica) para mitigar el sobreajuste (overfitting). El `'Dropout'` desactiva aleatoriamente un porcentaje de neuronas durante el entrenamiento, forzando a la red a aprender representaciones más robustas.
 - **Capa Densa Final (`'nn.Linear'`):** Una capa completamente conectada (`'nn.Linear'`) que tomará la salida final de la RNN (generalmente el estado oculto de la última capa recurrente en el último paso de tiempo o una agregación de ellos) y reducirá la dimensionalidad a una única salida para la clasificación binaria (ej. 0 para negativa, 1 para positiva).
2. **Despliegue de la Estructura y Parámetros:** Una vez definida la clase del modelo, instancienlo y desplieguen su estructura utilizando `'print(model)'`. Adicionalmente, impriman el desglose de los parámetros entrenables por capa, lo cual es útil para entender la complejidad del modelo y diagnosticar posibles problemas. Pueden iterar sobre `'model.named_parameters()'` para obtener esta información.

Paso 4: Configuración, Entrenamiento y Validación

Esta fase cubre el ciclo de entrenamiento del modelo, incluyendo la selección de la función de pérdida, el optimizador y la lógica de validación.

1. **Función de Pérdida y Optimizador:** Seleccionen la función de pérdida ideal para problemas de clasificación binaria. Se recomienda `'nn.BCEWithLogitsLoss'` en PyTorch, ya que combina la función Sigmoide y la Entropía Cruzada Binaria en una sola operación numéricamente estable. Elijan un optimizador adecuado, como `'Adam'`, que es comúnmente utilizado y ofrece buen rendimiento en una amplia gama de tareas.
2. **Ciclo de Entrenamiento:** Diseñen y ejecuten el ciclo principal de entrenamiento. Para cada época, asegúrense de:
 - **Limpiar Gradientes:** Antes de cada `'backward pass'`, limpien los gradientes acumulados de la iteración anterior (`'optimizer.zero_grad()'`).
 - **Forward Pass:** Pasen los datos de entrada a través del modelo para obtener las predicciones.
 - **Backward Pass:** Calculen los gradientes de la función de pérdida con respecto a los parámetros del modelo (`'loss.backward()'`).
 - **Gradient Clipping:** Apliquen una técnica de mitigación de explosión del gradiente como el `'Gradient Clipping'` (ej. `'nn.utils.clip_grad_norm_'`). Esto ayuda a prevenir que los gradientes crezcan exponencialmente y desestabilicen el entrenamiento [3].
 - **Actualizar Parámetros:** Actualicen los pesos del modelo utilizando el optimizador (`'optimizer.step()'`).

3. **Evaluación y Guardado del Modelo:** Al finalizar cada época, evalúen la métrica de pérdida y precisión en el conjunto de validación. Implementen una condicional para guardar automáticamente los pesos del modelo (`'state_dict'`) que haya logrado el mejor desempeño en el conjunto de validación (`'torch.save()'`). Esto asegura que, al final del entrenamiento, se disponga del modelo con la mejor capacidad de generalización.

Paso 5: Pruebas (Testing) y Métricas de Desempeño

Una vez entrenado, el modelo debe ser evaluado en un conjunto de datos no visto para medir su rendimiento real.

1. **Carga y Modo de Evaluación:** Instancien el modelo nuevamente y carguen los mejores pesos almacenados en el Paso 4. Pongan el modelo en modo de evaluación (`'eval()'`) para desactivar `'Dropout'` y `'BatchNorm'` (si se usaran), asegurando un comportamiento consistente durante la inferencia.
2. **Desactivación de la Gráfica Computacional:** Desactiven el cálculo de la gráfica computacional (`'torch.no_grad()'`) durante la fase de inferencia. Esto optimiza el cómputo al no almacenar gradientes, lo que es innecesario en la fase de prueba.
3. **Generación de Predicciones y Reporte de Métricas:** Generen predicciones para el conjunto completo de prueba (`'test set'`). A partir de estas predicciones y las etiquetas reales, calculen un reporte detallado que incluya las métricas de **Precision**, **Recall**, **F1-Score** y el **Accuracy** global de la evaluación. Estas métricas son cruciales para entender el rendimiento del modelo, especialmente en problemas de clasificación binaria donde la distribución de clases puede ser desequilibrada.

Paso 6: Visualizaciones Requeridas

Las visualizaciones son herramientas poderosas para comprender el comportamiento del modelo y diagnosticar problemas. Generen código para mostrar de forma clara las siguientes gráficas que soporten su análisis:

1. **Curvas de Entrenamiento:** Grafiquen comparativamente la **Función de Pérdida** (Train vs Val Loss) y la **Precisión** (Train vs Val Accuracy) en función de las épocas. Estas curvas son esenciales para identificar posibles comportamientos de sobreajuste (overfitting) o subajuste (underfitting).
2. **Matriz de Confusión:** Desplieguen un mapa de calor (`'heatmap'`) de la matriz de confusión derivada de las predicciones del Paso 5. La matriz de confusión permite visualizar el número de verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, ofreciendo una visión detallada de los tipos de errores que comete el modelo.
3. **Monitoreo de Gradientes:** Creen una gráfica que muestre la evolución de la **Norma L2 de los gradientes por capa** (ej. `'embedding'`, `'LSTM cap. 1'`, `'LSTM cap. 2'`, `'Linear'`) durante los primeros pasos del entrenamiento. Esta visualización es fundamental para diagnosticar si el flujo del gradiente es normal o si tiende a la explosión (`'exploding gradients'`) o al desvanecimiento (`'vanishing gradients'`). Pueden acceder a los gradientes de los parámetros de cada capa después del `'backward pass'` y antes del `'optimizer.step()'`.

4. Preguntas de Reflexión y Análisis

Estas preguntas están diseñadas para fomentar un análisis más profundo del laboratorio y los conceptos subyacentes.

Gestión del Vocabulario: Al construir el diccionario `'word2idx'` y filtrar palabras de baja frecuencia, ¿cómo impacta esta decisión en la generalización del modelo frente a posibles problemas de palabras fuera del vocabulario (Out-Of-Vocabulary o tokens `'<UNK>'`) en el conjunto de prueba? ¿Qué estrategias adicionales podrían emplearse para manejar eficazmente los tokens `'<UNK>'`?

Efectos del Padding: Al aplicar padding para obtener secuencias de longitud uniforme, el modelo procesa múltiples tokens `'<PAD>'`. Si no se implementa un mecanismo de enmascaramiento (`'masking'`) en la capa recurrente o en la función de pérdida, ¿cómo afecta matemáticamente este relleno al cálculo de los gradientes y a la actualización del estado oculto final? Describa las implicaciones y posibles soluciones.

LSTM vs. GRU: Considerando la arquitectura de la red, ¿cuál es la diferencia fundamental en el diseño de las compuertas (`'gates'`) entre una celda LSTM y una GRU? En términos empíricos, ¿qué diferencias en tiempos de entrenamiento y desempeño observarían al intercambiar estas celdas para este corpus en particular y por qué?

Profundidad y Jerarquía: La instrucción específica utilizar al menos 2 capas recurrentes apiladas. ¿Qué tipo de representaciones o características del lenguaje se espera que aprenda la segunda capa recurrente que la primera capa (que recibe directamente los embeddings) no es capaz de capturar? ¿Cómo contribuye esta jerarquía a la capacidad del modelo para entender el contexto y las relaciones a largo plazo en el texto?

Estabilidad Numérica: El modelo requiere compilarse con `nn.BCEWithLogitsLoss` para la salida binaria. ¿Cuál es la ventaja de estabilidad numérica al calcular el gradiente combinando internamente la función Sigmoide y la Entropía Cruzada Binaria, en lugar de utilizar una activación Sigmoide seguida de `nn.BCELoss`? Explique el problema que `nn.BCEWithLogitsLoss` ayuda a mitigar.

Diagnóstico del Gradiente: Al analizar la gráfica de la evolución de la Norma L2 de los gradientes, ¿cómo identificarían visualmente en las curvas de las primeras capas (como el embedding o la capa LSTM 1) que está ocurriendo un desvanecimiento (`vanishing gradient`)? ¿Por qué la técnica de `Gradient Clipping` previene que la red colapse en caso de explosión del gradiente?

Análisis de Errores (Matriz de Confusión): Si el mapa de calor de la matriz de confusión revela que el modelo tiene una alta tasa de falsos positivos (clasificando reseñas negativas como positivas), ¿qué métrica entre Precision y Recall se vería más afectada en la clase negativa y qué ajustes arquitectónicos o de datos propondrían para mitigarlo?

Dinámica de Aprendizaje: Observando las curvas de entrenamiento (Train vs Val Loss), ¿en qué punto exacto de las épocas determinarían que el modelo ha comenzado a memorizar el ruido del conjunto de entrenamiento (`overfitting`) en lugar de generalizar, y cómo interactúa la capa de regularización `nn.Dropout` para retrasar este fenómeno?